# Leitura de Arquivos Wavefront OBJ em C

## 1 Introdução

O formato `OBJ` foi criado pela empresa Wavefront Technologies para representar modelos geométricos tridimensionais. Atualmente é um dos formatos mais utilizados para intercâmbio de modelos entre aplicações de modelagem, CAD e motores gráficos.

O formato é baseado em texto ASCII, o que facilita sua leitura e geração. Cada linha do arquivo descreve um elemento da geometria. A seguir, iremos utilizar o conhecimento adquirido em disciplinas (ED1, OAD, etc) para implementar a leitura de um OBJ em C.

## 2 Estrutura Básica

Um arquivo OBJ simples pode conter apenas vértices e faces.

### 2.1 Vértices

Linhas iniciadas por `v` definem coordenadas tridimensionais:

```
v 0.0 0.0 0.0
v 1.0 0.0 0.0
v 0.0 1.0 0.0
```

Cada linha possui:

$$\texttt{v}\ x\ y\ z$$

onde:

- $x$: coordenada X;
- $y$: coordenada Y;
- $z$: coordenada Z.

### 2.2 Faces

Linhas iniciadas por `f` definem polígonos através de índices para os vértices:

```
f 1 2 3
```

Neste exemplo, a face é formada pelos vértices 1, 2 e 3. Os índices poderão ser quaisquer que existam dentro do arquivo, sendo a linha de face característica

$$\texttt{f}\ v_1\ v_2\ v_3 ... v_n$$

Uma característica importante do formato OBJ é que os índices dos vértices nas linhas "f"começam em **1** e não em 0.

## 3 Exemplo

O arquivo abaixo representa um triângulo:

```
v 0.0 0.0 0.0
v 1.0 0.0 0.0
v 0.0 1.0 0.0

f 1 2 3
```

## 4 Estruturas de Dados em C para vértices e faces triangulares

Podemos representar vértices e faces com as seguintes estruturas:

```c
typedef struct {
    float x, y, z;
} Vertice;

typedef struct {
    int v1, v2, v3;
} Face;
```

Assumiremos que todas as faces são triangulares.

## 5 Função de Leitura

A implementação a seguir lê apenas arquivos OBJ que tenham somente registros `v` e `f`.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_VERTICES 10000
#define MAX_FACES    10000

typedef struct {
    float x, y, z;
} Vertex;

typedef struct {
    int v1, v2, v3;
} Face;

int loadOBJ(const char *filename,
            Vertex *vertices,
            int *numVertices,
            Face *faces,
            int *numFaces)
{
    FILE *file = fopen(filename, "r");

    if (!file)
        return 0;

    char line[256];

    *numVertices = 0;
```

```
    *numFaces = 0;

    while (fgets(line, sizeof(line), file))
    {
        if (strncmp(line, "v ", 2) == 0)
        {
            Vertex v;

            if (sscanf(line,
                       "v %f %f %f",
                       &v.x,
                       &v.y,
                       &v.z) == 3)
            {
                vertices[*numVertices] = v;
                (*numVertices)++;
            }
        }
        else if (strncmp(line, "f ", 2) == 0)
        {
            Face f;

            if (sscanf(line,
                       "f %d %d %d",
                       &f.v1,
                       &f.v2,
                       &f.v3) == 3)
            {
                faces[*numFaces] = f;
                (*numFaces)++;
            }
        }
    }

    fclose(file);
    return 1;
}
```

## 6 Utilização

Exemplo de uso da função:

```
int main(void)
{
    Vertex vertices[MAX_VERTICES];
    Face faces[MAX_FACES];

    int numVertices;
    int numFaces;

    if (!loadOBJ("modelo.obj",
                 vertices,
                 &numVertices,
                 faces,
                 &numFaces))
    {
        printf("Erro ao abrir arquivo.\n");
```

```
        return 1;
    }

    printf("Vertices: %d\n", numVertices);
    printf("Faces: %d\n", numFaces);

    return 0;
}
```

## 7 Limitações

A função apresentada é uma versão bastante simplificada e suporta apenas:

- vértices (`v`);
- faces triangulares (`f`);
- índices simples do tipo `f 1 2 3`.

Arquivos OBJ reais frequentemente contêm:

- coordenadas de textura (`vt`);
- normais (`vn`);
- materiais (`mtl`);
- faces com mais de três vértices;
- definições do tipo `f v/vt/vn`.

Esses recursos exigem um parser mais completo. Caso tenha um OBJ mais complexo e queira convertê-lo, você poderá utilizar os comandos abaixo:

```
mesh_filter meu_objeto0.obj -center objeto.obj
ctmconv objeto.obj objtemp.obj --no-normals --no-texcoords
sed "s,//, ,g" objtemp.obj > objeto_converted.obj
rm objtemp.obj
```

Para isso, você deverá instalar os pacotes openctm e trimesh. Caso não faça questão de centralizar o objeto ou instalar o trimesh, você poderá suprimir a primeira linha do script.